

1) Design pseudocode to analyze a social network graph:

- 1. Find connected components**
- 2. Suggest shortest connection paths between users**

Pseudocode to Analyze a Social Network Graph

A social network can be represented as an **undirected graph**, where:

- **Vertex (V)** → Represents a user.
- **Edge (E)** → Represents a friendship or connection between two users.

The system should:

1. Find **connected components** (groups of mutually reachable users).
2. Find the **shortest connection path** between two users.

(a) Find Connected Components using DFS**Pseudocode**

ALGORITHM FindConnectedComponents(Graph G)

Mark all vertices as NOT_VISITED

 componentCount ← 0

 FOR each vertex v in G DO

 IF v is NOT_VISITED THEN

 componentCount ← componentCount + 1

 PRINT "Connected Component ", componentCount

 DFS(v)

 END IF

 END FOR

END ALGORITHM

PROCEDURE DFS(v)

 Mark v as VISITED

 PRINT v

 FOR each neighbor u of v DO

IF u is NOT_VISITED THEN

DFS(u)

END IF

END FOR

END PROCEDURE

Explanation

1. Initially, all users are marked as **not visited**.
2. Select an unvisited user.
3. Perform **Depth First Search (DFS)** to visit all users connected to that user.
4. All visited users form one **connected component**.
5. Repeat the process until all users are visited.

Example

If users are connected as:

A — B — C D — E

The connected components are:

Component 1: A, B, C

Component 2: D, E

(b) Suggest the Shortest Connection Path Between Users

For an unweighted social network graph, **Breadth First Search (BFS)** is used because BFS finds the shortest path in terms of the minimum number of connections.

Pseudocode

ALGORITHM ShortestPath(Graph G, Source, Destination)

Create an empty Queue Q

Mark all vertices as NOT_VISITED

parent[each vertex] ← NULL

Mark Source as VISITED

ENQUEUE(Q, Source)

WHILE Q is NOT EMPTY DO

current ← DEQUEUE(Q)

```

IF current = Destination THEN
    BREAK
END IF
FOR each neighbor of current DO
    IF neighbor is NOT_VISITED THEN
        Mark neighbor as VISITE
        parent[neighbor] ← current
        ENQUEUE(Q, neighbor)
    END IF
END FOR
END WHILE
IF Destination is NOT_VISITED THEN
    PRINT "No connection path exists"
ELSE
    Reconstruct the path using parent[]
    PRINT "Shortest connection path"
END IF
END ALGORITHM

```

Example

A — B — D

\ /

C — E

To find the shortest connection from **A to D**, BFS may give:

A → B → D

Result

Thus, the social network analysis system:

- Uses **DFS** to identify different connected communities.
- Uses **BFS** to suggest the shortest connection path between users.
- Helps identify isolated groups and improve user connectivity.

2) “Design a task scheduling system for a real-world application

(e.g., project management) **Requirements:**

1. **Model the problem as a directed graph**
2. **Ensure no cyclic dependencies exist**
3. **Generate a valid execution order of tasks**

A task scheduling system can be represented using a **Directed Graph**, where:

- **Vertices** represent tasks.
- **Directed edges** represent task dependencies.
- A **cycle detection algorithm** is used to ensure no circular dependency exists.
- **Topological Sorting** is used to generate a valid execution order.

Pseudocode

ALGORITHM TaskScheduling(G)

Calculate indegree of each task

Add all tasks with indegree = 0 to Queue

WHILE Queue is not empty DO

task ← DEQUEUE(Queue)

PRINT task

FOR each dependent task of task DO

Decrease its indegree by 1

IF indegree = 0 THEN

 ENQUEUE(task)

 END IF

END FOR

END WHILE

IF all tasks are processed THEN

 PRINT "Valid execution order generated"

ELSE

 PRINT "Cycle detected"

END IF

END ALGORITHM

Conclusion: Thus, a **Directed Acyclic Graph (DAG)** and **Topological Sorting** help schedule tasks in the correct order without cyclic dependencies.

3) Write pseudocode for constructing an MST using Kruskal's Algorithm

A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

1. All zones must be connected
2. Total installation cost must be minimized
3. No redundant connections (no cycles) allowed
4. The network must remain fully connected

A city fiber-optic network can be represented as a weighted undirected graph, where:

- Vertices represent city zones.
- Edges represent possible cable connections.
- Weights represent installation costs.

The objective is to connect all zones with minimum total cost without forming cycles.

Pseudocode

ALGORITHM KruskalMST(G)

Sort all edges in increasing order of weight

MST $\leftarrow$ Empty Set

FOR each edge (u, v) in sorted edges DO

IF adding (u, v) does not form a cycle THEN

Add (u, v) to MST

END IF

IF MST contains $V - 1$ edges THEN

BREAK

END IF

END FOR

PRINT MST

END ALGORITHM

How the Constraints Are Satisfied

1. **All zones are connected:**
The MST connects every vertex in the graph.
2. **Minimum installation cost:**
Kruskal's algorithm selects edges starting from the lowest cost.
3. **No redundant connections:**
An edge is added only when it does not create a cycle.
4. **Network remains fully connected:**
The algorithm continues until exactly $V - 1$ edges connect all zones.

Advantages

- Provides the **minimum total installation cost**.
- Avoids unnecessary cable connections.
- Prevents cycles and redundant paths.
- Suitable for large network planning problems.
- Efficient when implemented using the **Union-Find** data structure.

Conclusion

Thus, **Kruskal's Algorithm** is an efficient method for designing a minimum-cost fiber-optic network. By selecting the lowest-cost connections while avoiding cycles, it constructs a **Minimum Spanning Tree** that connects all city zones with **minimum cost, no redundancy, and full network connectivity**.